Refuges

Rapport final du projet

Conception d'une application web trois tiers,
déploiement sur VM et Docker, tests de charge JMeter.

Akrem SOUSSI et Rémi CORTIAL

Polytech Marseille - Aix-Marseille Université
DEV 1 - Application trois tiers
Enseignant : Mohamed Quafafou

Une application communautaire pour rechercher des refuges, partager des publications et organiser des sorties en montagne.

Ce rapport rassemble la démarche, les choix techniques, les preuves principales et les limites des mesures. Les procédures détaillées et les résultats bruts accompagnent le document dans le dépôt.

Sommaire

1. Objectifs et organisation du projet 3

2. De la maquette Figma à l’application 4

3. Analyse fonctionnelle et règles métier 6

4. Architecture et circulation des données 10

5. Deux parcours vérifiés dans l’application 11

6. Déploiement sur trois machines virtuelles 13

7. Docker : déploiement et comparaison 15

8. JMeter : protocole, résultats et interprétation 17

9. Bilan, limites et pistes d’amélioration 21

10. Documents et preuves associés 22

Lire les résultats

Nous distinguons trois niveaux : le fonctionnement décrit dans le code et la recette historique, les parcours récemment observés dans le navigateur et les mesures instrumentées. Une observation sur un parcours ne prouve pas à elle seule le fonctionnement de toutes les fonctions.

Les mesures de référence de Rémi et la campagne comparative JMeter sont présentées séparément. Leurs machines, versions, périmètres mémoire et conditions de démarrage diffèrent. Les comparer directement donnerait une conclusion trompeuse.

1. Objectifs et organisation du projet

Nous avons développé Refuges pour réunir la consultation de refuges et les échanges entre randonneurs dans une même application. Le parcours principal part d'une recherche sur une carte et mène à une fiche pratique. Les communautés et les sorties ajoutent une dimension collaborative.

Une démarche en quatre étapes

Étape	Travail réalisé
Concevoir et développer	Définir les acteurs, les règles de gestion, la maquette et le modèle de données ; réaliser le client et l'API.
Déployer sur des VM	Séparer présentation, métier et données sur trois systèmes invités ; configurer et vérifier les échanges réseau.
Conteneuriser	Décrire les services avec Docker Compose, conserver les données dans des volumes et vérifier l'isolation.
Mesurer sous charge	Valider le scénario JMeter, étalonner l'injection, exécuter les paliers et interpréter débit, latence, erreurs et ressources.

Organisation du dépôt

Le dossier client/ contient l'interface React et TypeScript ; metier/ contient l'API FastAPI ; donnees/ contient le schéma et le peuplement MySQL. Les configurations VM sont dans deploiement/, celles de Docker dans docker/. Les plans, scripts et résultats JMeter sont regroupés dans charge/. Les analyses et preuves visuelles se trouvent dans docs/.

Nous avons utilisé l'IA pour nous aider à structurer les idées, réaliser la maquette et le code, préparer les configurations et analyser les résultats. Les documents ne présentent pas les propositions de l'IA comme des mesures : les chiffres retenus renvoient aux relevés archivés. Les prompts de conception, dont l'historique complet n'a pas été conservé, sont explicitement reconstitués dans la section suivante.

Données et ressources

Le jeu de données contient 4 652 refuges, 156 massifs, 6 communautés et 30 utilisateurs, dont 25 comptes de charge. Les informations des refuges proviennent de refuges.info. Les fonds de carte utilisent OpenStreetMap ; les attributions restent présentes dans l'application.

2. De la maquette Figma à l’application

Voir notre maquette Figma

Nous n’avons pas conservé l’historique complet de nos échanges avec l’IA. Cette présentation reconstitue notre démarche ; les exemples de prompts sont reformulés pour expliquer nos intentions et ne sont pas les messages exacts envoyés à l’époque.

Organiser les fonctionnalités

Avec Refuges, nous voulions permettre aux utilisateurs de trouver un refuge, de consulter ses informations et de préparer des sorties avec d’autres randonneurs. Nous avons organisé l’interface autour de trois entrées principales : la carte, les communautés et les sorties. Les pages de connexion, de profil et de modération complètent ces parcours.

L’IA nous a servi d’appui pour structurer ces besoins et réfléchir à leur présentation. Nos demandes portaient d’abord sur l’organisation des écrans, avant de détailler leur apparence.

> « Nous travaillons sur une application de randonnée avec une carte des refuges, des communautés et des sorties. Propose une organisation des écrans et un parcours pour chercher un refuge, consulter sa fiche puis préparer une sortie. Prévois aussi la connexion et la modération. »

Définir l’apparence de la maquette

Nous avons retenu une palette verte, des fonds clairs et des cartes blanches pour rappeler l’univers de la nature tout en gardant une interface lisible. Les boutons, les champs et les badges suivent les mêmes règles sur les différentes pages.

Notre interface reprend des principes courants des sites de cartographie et de randonnée : associer une carte à une liste de résultats, rendre la recherche visible et regrouper les informations pratiques dans une fiche. Nous n’avons pas conservé les références précises des sites consultés et ne pouvons donc pas attribuer ces inspirations à des sites particuliers.

> « Propose une interface sobre, dans un univers montagne et nature, avec une palette verte et une typographie lisible. Sur la page Carte, organise la recherche, les filtres et la liste des refuges sans surcharger l’écran. »

Nous avons ensuite affiné les demandes écran par écran : mettre les informations importantes en avant, alléger les éléments secondaires et conserver les mêmes composants. Cette approche permet de préciser ce que nous attendons de l’IA au lieu de lui demander simplement de produire une application entière.

> « Sur la fiche refuge, rends le nom, la localisation et les informations pratiques plus visibles. Reprends les boutons et les arrondis de la page Carte. Prévois un affichage sans photo et un message lorsqu’une recherche ne retourne aucun résultat. »

Choisir les ressources et intégrer le site

Le site final utilise la police **Inter**, intégrée avec Fontsource, et les icônes **Lucide**. Les cartes interactives reposent sur **Leaflet et React Leaflet**, avec les tuiles **OpenStreetMap**. Les informations sur les refuges sont attribuées à **refuges.info** dans les cartes.

Ces ressources sont identifiables dans le code de l'application. Nous ne disposons pas d'un inventaire équivalent pour les images de la maquette et ne revendiquons donc aucune banque d'images ni génération d'images particulière.

Pour passer de la maquette au site, nous avons utilisé React et TypeScript. Les couleurs, les tailles et les arrondis sont centralisés dans des fichiers de styles communs, dont les commentaires font référence à la maquette Figma. Cela permet de conserver une présentation cohérente entre les pages.

L'intégration ajoute aussi les comportements nécessaires : navigation, connexion, chargement des données de l'API et interactions avec la carte.

> « Intègre cette page en React et TypeScript en reprenant les styles de la maquette et les composants existants. Relie-la aux données de l'API et prévois les états de chargement, d'erreur et de liste vide. Signale les adaptations nécessaires par rapport à la maquette. »

Le rôle de l'IA dans notre travail

L'IA nous a accompagnés pour organiser les idées, proposer des solutions visuelles et aider à leur traduction en code. Notre rôle reste de préciser les besoins, de sélectionner les propositions adaptées au projet et de vérifier le résultat. La maquette donne une référence visuelle ; l'application doit ensuite rendre les parcours réellement utilisables.

3. Analyse fonctionnelle et règles métier

Objectif et architecture

Refuges est une application communautaire de randonnée. Elle permet de rechercher un refuge sur une carte, de consulter ses informations, de partager des publications dans des communautés et de préparer des sorties. La base contient 4 652 refuges issus de refuges.info. Les informations et les fonds OpenStreetMap sont accompagnés de leurs attributions.

L'application suit une architecture trois tiers. Le navigateur affiche le client React/TypeScript, servi par Nginx. Les requêtes /api/ sont transmises à FastAPI, qui applique les règles métier et accède à MySQL via SQLAlchemy. Le navigateur ne dialogue jamais directement avec la base. Les déploiements VM et Docker conservent cette séparation.

Acteurs et droits

Acteur	Fonctions accessibles	Limites principales
Visiteur	Connexion, inscription, confirmation d'email, récupération du mot de passe, conditions et statut du service.	Les pages applicatives nécessitent une session active. Un compte en attente reste limité à la confirmation.
Utilisateur (user)	Carte, refuges, communautés, publications, sorties, amis, profil, confidentialité et signalements.	Aucun accès à la modération. Modification de ses propres contenus uniquement.
Administrateur (moderator)	Fonctions utilisateur, traitement des signalements, gestion des communautés et des comptes ordinaires.	Ne peut pas changer les rôles ni agir sur les comptes administrateurs.
Administrateur supérieur (super_admin)	Fonctions précédentes, promotion/rétrogradation des modérateurs et gestion de leurs comptes.	Ne peut pas suspendre, supprimer ou rétrograder son propre compte.

Les restrictions sont appliquées par les gardes de routes du client et par les contrôles d'autorisation du serveur. Masquer un bouton ne remplace pas une vérification côté API : une requête interdite doit également être rejetée.

Principes de conception

La navigation principale regroupe Carte, Communautés et Sorties ; la modération apparaît pour les rôles autorisés. Les styles partagés utilisent Inter, un vert principal #21664a et des fonds clairs. Les formulaires et listes prévoient des états de chargement, d'erreur et de résultat vide.

La maquette sert de référence pour la structure et les composants. Le site ajoute les données réelles, les sessions, les contrôles métier et les interactions cartographiques. Notre utilisation de l'IA est décrite séparément dans la démarche Figma.

Cas d’utilisation : accès et consultation

CU1 - S’inscrire et se connecter

Acteur et déclencheur : un visiteur soumet le formulaire d’inscription ou de connexion.

Scénario nominal : le visiteur renseigne un pseudo unique, son email et un mot de passe d’au moins huit caractères, puis accepte les conditions. Le serveur conserve un condensat Argon2id du mot de passe, crée un compte en attente et un jeton de confirmation. La validation du lien active le compte. Une connexion valide crée une session serveur et un cookie refuges_session HttpOnly, SameSite=Lax.

Erreurs : pseudo ou email déjà utilisé : HTTP 409 ; jeton expiré ou déjà utilisé : HTTP 400 ; identifiants erronés : HTTP 401 avec le même message générique ; compte suspendu : HTTP 403. Un compte non confirmé reste sur l’écran d’attente.

CU2 - Réinitialiser son mot de passe

Acteur et déclencheur : un visiteur sélectionne « Mot de passe oublié » et renseigne son email.

Scénario nominal : pour un compte admissible, le serveur génère un lien temporaire valable une heure. La validation du nouveau mot de passe remplace son condensat et révoque toutes les anciennes sessions.

Erreurs : un lien invalide ou expiré est rejeté en HTTP 400. La demande initiale reçoit une réponse identique, que l’adresse soit connue ou non, afin de ne pas révéler les comptes inscrits.

CU3 - Rechercher et consulter un refuge

Acteur et déclencheur : un utilisateur connecté ouvre la carte, déplace son emprise ou saisit une recherche.

Scénario nominal : le client demande les refuges correspondant à l’emprise et aux filtres. Les marqueurs sont regroupés et une liste accompagne la carte. La sélection d’un refuge donne accès à sa fiche : altitude, capacité, gardiennage, description, accès, photos disponibles, publications et sorties associées.

Erreurs : une recherche sans résultat affiche un état vide ; un identifiant inexistant renvoie HTTP 404. L’absence de photographie est représentée par un emplacement explicite.

Exemple de parcours réalisé : carte → recherche « Pelvoux » → sélection → fiche du refuge. Les captures du parcours montrent le résultat dans l’application locale.

Cas d’utilisation : communauté et administration

CU4 - Publier dans une communauté

Acteur et déclencheur : un utilisateur membre ouvre « Nouveau post ».

Scénario nominal : il choisit la communauté, saisit son texte et peut associer un refuge, jusqu’à quatre photos et une trace GPX. Il choisit une visibilité publique ou limitée aux amis. Les images sont réencodées en WebP et nettoyées de leurs métadonnées EXIF. Le serveur calcule la distance de la trace et son dénivelé positif lorsque des altitudes sont disponibles.

Erreurs : une publication vide ou une image invalide est rejetée en HTTP 422 ; publier dans une communauté sans en être membre est interdit en HTTP 403.

Parcours illustré : communautés → Randonneurs des Écrins → nouveau post → publication textuelle → vérification après rechargement. Cette démonstration ne couvre pas l’import de photo ou de GPX.

CU5 - Organiser et rejoindre une sortie

Acteur et déclencheur : un utilisateur crée une sortie ou ouvre une sortie pour y participer.

Scénario nominal : l’organisateur définit le titre, le départ, une capacité de 2 à 50 participants, la visibilité et au moins une étape. Une trace peut être ajoutée. L’organisateur devient le premier participant ; les autres utilisateurs admissibles peuvent rejoindre ou quitter la sortie.

Erreurs : une sortie complète ou passée refuse une nouvelle participation en HTTP 409. Une sortie sur invitation est masquée ou refusée aux utilisateurs non invités. Les modifications sont contrôlées selon l’identité et les droits de l’utilisateur.

CU6 - Modérer les contenus et les comptes

Acteur et déclencheur : un modérateur ou un administrateur supérieur ouvre l’espace de modération.

Scénario nominal : il examine les signalements, traite les contenus concernés et gère les communautés. La suspension d’un compte révoque ses sessions et empêche sa reconnexion. L’administrateur supérieur peut aussi attribuer ou retirer le rôle de modérateur. Les actions sont consignées dans le journal de modération.

Erreurs : une action dépassant les droits du modérateur est rejetée en HTTP 403. Une action interdite sur son propre compte est rejetée en HTTP 409.

Vérification fonctionnelle

Le cahier de recette décrit les cas nominaux, les erreurs et les contrôles des quatre profils. Les deux parcours photographiés le 26 octobre complètent cette recette historique ; ils ne constituent pas une nouvelle exécution de tous les tests. Les essais de charge JMeter font l’objet d’un rapport distinct.

Règles de gestion et modèle relationnel

Règle	Traduction dans les données et le métier
Identité et statut	users impose l'unicité du pseudo et de l'email. Rôle et statut sont limités aux valeurs prévues.
Authentification	Mots de passe sous forme de condensats Argon2id ; jetons de session et d'email sous forme d'empreintes SHA-256. L'auteur d'une action est déterminé par la session serveur.
Refuges et massifs	refuges.external_id est unique. Un refuge peut être lié à un massif et à plusieurs photos ordonnées.
Communautés	Le slug est unique. community_refuges représente l'association multiple entre communautés et refuges.
Adhésions et réactions	Les clés composées de community_members et post_likes empêchent les doublons d'adhésion ou de réaction.
Amitiés	Une contrainte empêche les doublons dans un même sens ; le métier vérifie aussi la relation inverse. Les audiences règlent l'accès aux informations privées.
Sorties	Un organisateur, des étapes ordonnées et des participants uniques ; capacité, date et invitations sont contrôlées par le serveur.
Suppression et journal	Les clés étrangères assurent les suppressions dépendantes prévues. Le journal de modération est conservé, avec un acteur nullable si son compte disparaît.

Correspondance entre maquette et application

Écrans Figma	Pages et routes principales
01, 02, 02b - Accès	/connexion, /inscription, /inscription/confirmation ; confirmation d'email et récupération du mot de passe.
03, 04 - Refuges	/carte, /refuges/:id.
05, 05b, 05c, 06, 06b - Profils	/profil, /parametres/amis, /parametres/confidentialite, /u/:username ; affichage selon les droits de visibilité.
07, 08, 09 - Communautés	/communautes, /communautes/:slug, /posts/nouveau.
10, 11, 12 - Sorties	/sorties, /sorties/:id, /sorties/nouvelle, modification d'une sortie.
13 à 16 - Modération	/moderation, signalements, communautés et comptes ; accès selon le rôle.

Maquette Figma. Les identifiants détaillés des écrans, les règles SQL et les scénarios étendus restent disponibles dans l'annexe d'analyse. Les diagrammes décrivent la base, les cas d'utilisation, les classes, les séquences et l'architecture. Les exemples API et la recette complètent le dossier.

4. Architecture et circulation des données

Trois responsabilités distinctes

Le navigateur exécute le client React. Nginx distribue ses fichiers statiques et transmet les requêtes /api/ à FastAPI. Le métier identifie l'utilisateur, contrôle ses droits, applique les règles et interroge MySQL avec SQLAlchemy. La base conserve les données et leurs contraintes d'intégrité.

Couche	Technologies et responsabilité
Présentation	React, TypeScript, Vite, Leaflet : navigation, formulaires, cartes et états de l'interface.
Métier	FastAPI et SQLAlchemy : validation, authentification, autorisations et orchestration des traitements.
Données	MySQL : stockage relationnel, unicité, clés étrangères et associations.

Exemple : connexion puis consultation

Le client transmet les identifiants à l'API. Après vérification du condensat du mot de passe, le serveur crée une session et remet un cookie HttpOnly. La consultation suivante transmet ce cookie ; l'API résout la session, vérifie l'accès, charge les refuges puis renvoie une réponse JSON. Le client met à jour la liste et les marqueurs. Le navigateur n'accède jamais directement à MySQL.

Modèle de données

Les principaux ensembles concernent les utilisateurs et sessions, les refuges et photos, les communautés et publications, les amitiés, les sorties et la modération. Les associations disposent de contraintes évitant les doublons. Le schéma complet comporte 20 tables en incluant le suivi Alembic ; le diagramme applicatif en présente 19. Les modèles ORM comprennent 18 classes persistantes, certaines associations étant représentées directement par des tables.

Les diagrammes du projet détaillent les cardinalités, les classes, les cas d'utilisation et les séquences. Ils restent consultables en SVG pour conserver leur lisibilité au zoom.

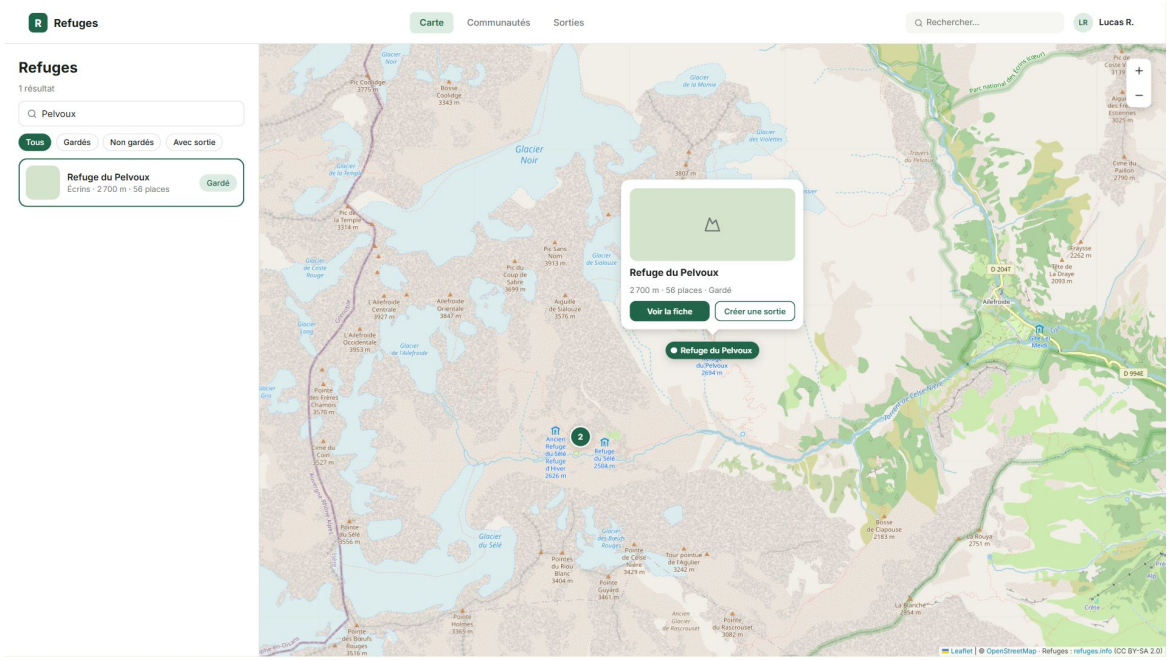
Médias et confidentialité

Les images envoyées sont converties en WebP et nettoyées de leurs métadonnées EXIF. Les photos de refuges sont importées par des scripts, puis servies localement ; ouvrir une fiche ne déclenche pas un import externe. Les contrôles de visibilité et de propriété sont effectués côté serveur, en complément de l'interface.

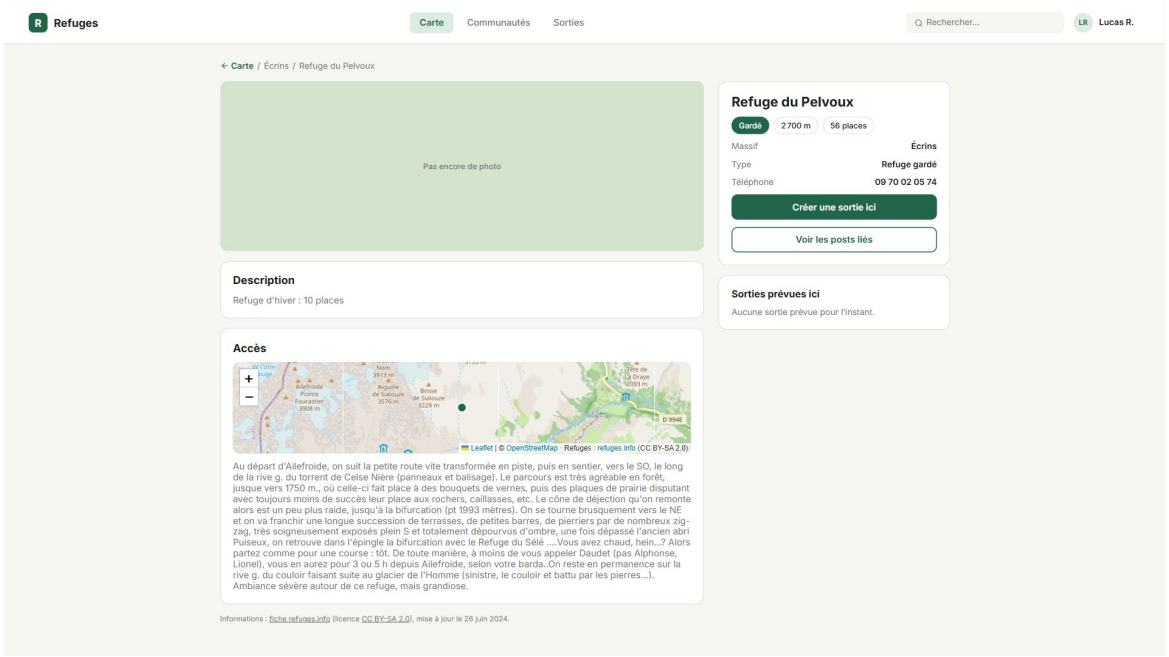
5. Deux parcours vérifiés dans l’application

Rechercher et consulter le refuge du Pelvoux

La session du compte de démonstration Lucas Randonneur était ouverte. Nous avons recherché « Pelvoux », sélectionné le résultat, puis ouvert la fiche. Elle indique notamment une altitude de 2 700 m et une capacité de 56 places. L’absence de photo est représentée explicitement.



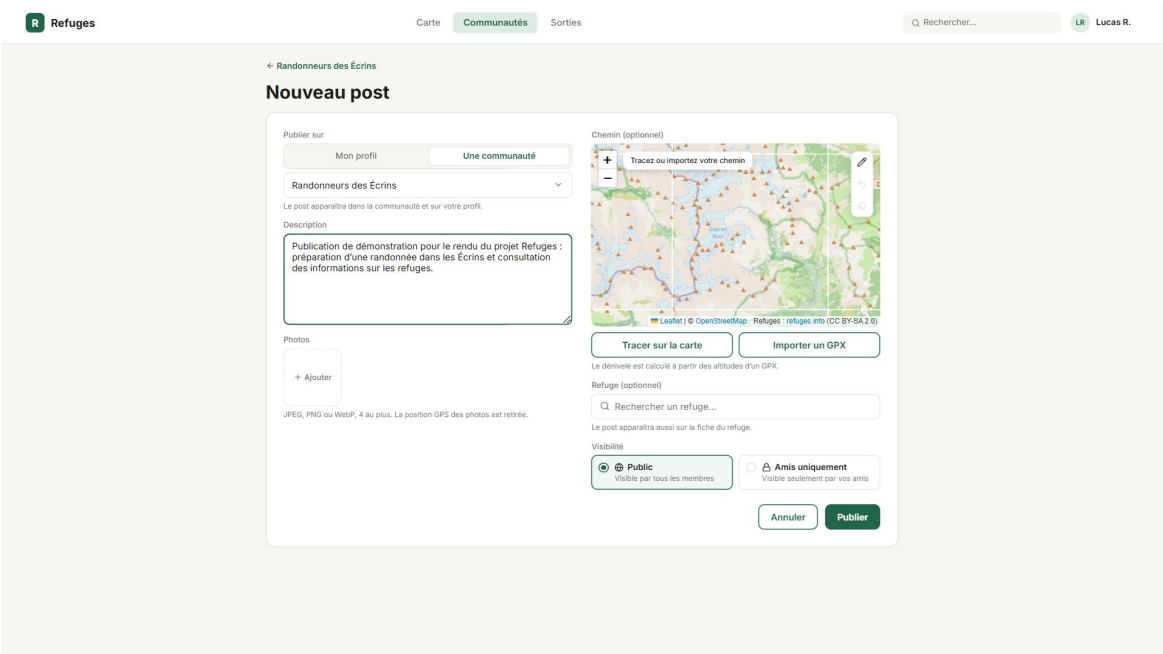
Recherche du Pelvoux : un résultat et accès à la fiche.



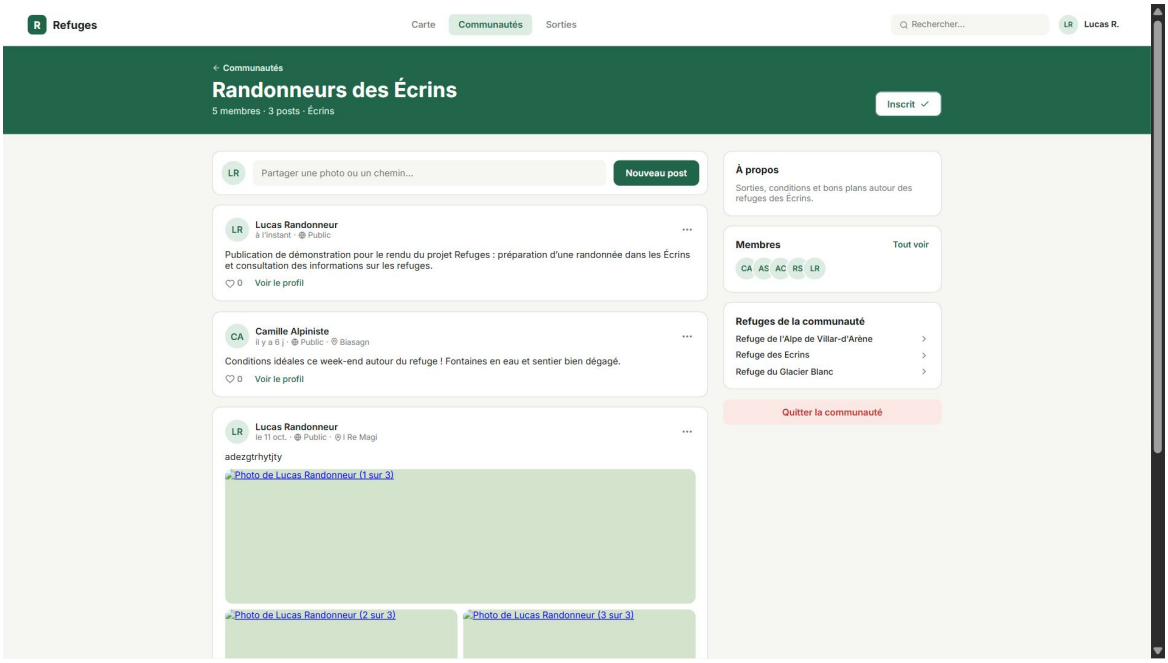
Fiche du refuge : informations pratiques et état sans photographie.

Publier dans la communauté des Écrins

Le compte était déjà membre des Randonneurs des Écrins. Nous avons rédigé un message de démonstration, conservé sa visibilité publique puis publié. Le compteur est passé de deux à trois publications et un rechargement a confirmé la persistance du message.



Formulaire rempli avant l'envoi du message.



Message conservé après publication et rechargement.

Ce deuxième parcours couvre le texte, pas l'envoi d'une photo ou d'un GPX. Les anciennes images manquantes visibles dans la capture ont ensuite été restaurées depuis leurs fichiers originaux ; leurs URL ont été contrôlées en HTTP 200. Les huit captures et les explications complètes sont dans le dossier des parcours.

6. Déploiement sur trois machines virtuelles

Préparation et topologie

Le déploiement repose sur trois VM Ubuntu Server 24.04 LTS issues d'un socle commun. Les clones reçoivent des adresses MAC et des identités machine distinctes. La configuration vérifiée pour les nouvelles mesures est de 1 vCPU et 2 048 Mo par VM.

Machine	Adresse interne	Rôle et accès
vm-proxy	10.10.10.10/24	Nginx ; réseau interne et NAT ; port hôte 8080 redirigé vers le port 80.
vm-metier	10.10.10.20/24	FastAPI sur le port 8000 ; réseau interne uniquement après installation.
vm-donnees	10.10.10.30/24	MySQL sur le port 3306 ; réseau interne uniquement après installation.

Le réseau interne VirtualBox est nommé intnet-app. Le NAT est activé ponctuellement pour installer les dépendances des couches internes, puis désactivé. Le proxy reste le point d'entrée web depuis l'hôte.

Installation des services

Nous installons MySQL, limitons son écoute à l'adresse interne et importons le schéma et le peuplement. Le compte SQL applicatif est restreint à la machine métier. Le pare-feu autorise le port 3306 depuis cette machine. Le backend est installé dans un environnement Python et démarré par systemd. Ses paramètres de connexion sont externalisés. Nginx sert le client compilé et transmet /api/ vers le métier.

Les contrôles portent d'abord sur la connectivité, puis sur la santé de l'API et de la base. Les snapshots permettent de conserver un socle propre et des étapes de configuration. Les fichiers Netplan, Nginx et systemd accompagnent la procédure VM.

Vérifier les communications autorisées et interdites

Origine	Proxy :80	Métier :8000	MySQL :3306
Hôte Windows	Oui, via 8080	Bloqué	Bloqué
Proxy	Sans objet	Oui	Bloqué
Métier	Oui	Sans objet	Oui
Données	Oui	Oui	Sans objet

Les trois refus essentiels sont l'accès direct de l'hôte au métier, l'accès direct de l'hôte à la base et l'accès du proxy à la base. La matrice ne signifie pas que tous les autres flux sont interdits : les flux sortants des VM internes indiqués comme accessibles restent autorisés dans la configuration observée.

Les captures depuis les quatre origines et celles des snapshots sont conservées dans docs/captures/. Elles complètent la configuration écrite par des observations effectives.

Export et remise

Les trois OVA récents totalisent environ 2,88 Go : 910 198 784 octets pour les données, 1 039 155 200 pour le métier et 927 240 192 pour le proxy. Leurs manifestes internes et leurs empreintes SHA-256 ont été vérifiés. Cette vérification d'intégrité ne remplace pas un nouvel import complet sur un autre poste.

Les archives sont conservées localement dans ova/ et disponibles dans la release du rendu. Le guide des OVA précise les empreintes et les paramètres à contrôler lors de l'import. Un export OVA ne contient pas l'historique des snapshots.

Portée des anciennes mesures

Les premières valeurs de l'étape 10 sont conservées comme historique. Leur allocation mémoire et leur méthode de contrôle HTTP diffèrent des nouvelles mesures instrumentées. Nous ne les fusionnons pas avec les séries présentées ci-après. Le temps initial de préparation des VM n'a pas été chronométré ; aucune durée inventée ne lui est attribuée.

7. Docker : déploiement et comparaison

Construction et fonctionnement

Nous conservons les trois couches : Nginx, FastAPI et MySQL. Docker Compose décrit les services, les réseaux et les volumes. Le client est compilé avec npm ci dans une étape de construction ; Nginx ne conserve que le résultat. Le métier utilise requirements.lock et les images de base sont fixées par digests. Les paramètres privés sont dans .env.docker, exclu de Git.

Le script docker/demarrer.sh construit les images, lance les services, attend leur santé puis applique le filtrage hôte. Le proxy est publié sur 127.0.0.1:8080. Le réseau web relie proxy et métier ; le réseau data relie métier et base. Les ports 8000 et 3306 ne sont pas publiés. La procédure Docker contient les commandes de reproduction.

Isolation vérifiée

Origine	Proxy :80	Métier :8000	MySQL :3306
Windows	Oui via 8080	Bloqué	Bloqué
Hôte WSL	Oui via 8080	Bloqué après filtrage	Bloqué après filtrage
Proxy	Sans objet	Oui	Bloqué
Métier	Oui	Sans objet	Oui
Données	Bloqué	Oui	Sans objet

Les tests utilisent les IP effectives, avec des contrôles positifs pour prouver que les services fonctionnent. Sous le moteur Linux natif, l'absence de publication ne suffisait pas à interdire l'accès direct depuis WSL : une chaîne iptables limitée aux réseaux du projet bloque les ports privés 8000, 3306 et 33060.

Mesures de référence sur le poste de Rémi

Trois essais par déploiement, jamais simultanés, puis huit secondes de stabilisation et dix consultations authentifiées par essai. Les 60 consultations répondent HTTP 200. Le démarrage se termine lorsque /api/health confirme aussi la base. Les valeurs ci-dessous sont les médianes.

Grandeur	Trois VM	Trois conteneurs
Démarrage	122,80 s	14,59 s
Mémoire au repos	3 073,52 MiB, processus VM	4 671,84 MiB, WSL complet
Disque relevé	25,36 GiB, VDI et snapshots	559,93 MiB, images et volumes
Latence de consultation	64,80 ms	23,70 ms

Interpréter les écarts

Le démarrage est 8,42 fois plus court et la latence 2,73 fois plus faible dans ce relevé Docker. Les VM démarrent trois systèmes invités ; les conteneurs réutilisent le noyau Linux déjà actif dans WSL. Le moteur Docker reste démarré entre ces essais, contrairement au protocole JMeter présenté ensuite.

Les trois VM disposent chacune de 1 vCPU et 2 Go, tandis que cette configuration Compose ne fixe pas de quotas. Les versions et la révision invitée des VM ne sont pas attestées identiques. Les écarts décrivent donc ces installations au repos ; ils ne mesurent pas le seul effet de la conteneurisation.

La mémoire Docker est celle de tout vmmemWSL, avec d'autres processus et des caches. Le relevé avant campagne était déjà de 4 700,02 MiB. La mémoire VM est une somme de processus VirtualBox : aucun gain mémoire net n'est déduit de ces périmètres différents. De même, le disque Docker exclut moteur, caches et distribution WSL, et certaines couches d'images peuvent être partagées.

Persistance et reproductibilité

Nous avons créé un marqueur dans une table dédiée, recréé les trois conteneurs puis retrouvé ce marqueur et les mêmes compteurs métier. Les identifiants des conteneurs ont changé, les volumes sont restés identiques ; la table de test a ensuite été retirée. La base contient 4 652 refuges, 156 massifs, 6 communautés et 30 utilisateurs. Le volume uploads conserve les médias séparément du SQL.

La reconstruction sans cache prend 41,26 s, puis l'initialisation d'un volume vide et l'attente de services sains prennent 52,50 s : total 93,76 s. Le moteur et les images de base sont déjà présents ; leur installation est exclue. La préparation initiale des VM n'a pas été chronométrée. Elle nous a paru plus longue, mais aucun gain chiffré n'est annoncé.

Ce que les conteneurs changent

Les conteneurs partagent un noyau et reposent sur des espaces de noms et des groupes de contrôle. Ils ne fournissent pas trois noyaux invités indépendants. Le moteur et le noyau constituent une dépendance commune ; sous Windows, WSL ajoute aussi un environnement Linux. Cette solution simplifie la description et la reconstruction des services, mais ne remplace pas toutes les propriétés d'isolation des VM.

Les versions, matrices avant et après filtrage, contrôles de persistance et mesures brutes sont dans les preuves Docker. Le détail de cette comparaison est conservé dans le document Markdown.

8. JMeter : protocole, résultats et interprétation

Protocole

Même poste Windows, même code applicatif, mêmes fichiers SQL et comptes figés, même plan JMeter 5.6.3 / Java 8u511. Chaque couche dispose de 1 CPU et 2 Gio de RAM (système invité compris pour les VM). Un seul déploiement fonctionne à la fois ; Docker Desktop est arrêté pendant les mesures VM. L'ordre des campagnes est VM r1, Docker r1, Docker r2, VM r2. Une sauvegarde privée et une réinitialisation SQL précèdent chaque série.

Une itération ouvre une session puis consulte 20 refuges authentifiés ; assertions HTTP, contenu et session, attente de 500 à 1 000 ms avant chaque requête. Paliers : 1, 5, 10, 25, 50 et 100 utilisateurs. Montée 10 s, fenêtre mesurée 60 s après le marqueur + 11 s, pause 10 s entre paliers. Les requêtes sont retenues par leur heure de départ ; leur durée complète est conservée. Les contrôles positif et mot de passe incorrect sont archivés pour chaque série.

Synthèse

Grandeur	VM	Docker
Meilleur débit observé sans erreur	3,77 req/s (10 u.)	5,48 req/s (10 u.)
p95 moyen à 10 utilisateurs	3,79 s	2,50 s
Erreurs moyennes à 100 utilisateurs	100,00 %	99,50 %
Premier palier proche du plateau (1)	10 utilisateurs	5 utilisateurs
Composant limitant observé	Backend : CPU / pool	Backend : CPU / pool
Démarrage complet, médiane de 3	63,53 s	26,86 s
Somme RSS hôte au repos, médiane	2229 Mio	3472 Mio
Pic mémoire backend observé	972 Mio	869 Mio

La somme RSS mesure les processus hôtes identifiés, avec possibles pages partagées ; elle ne représente pas la RAM allouée. Le démarrage VM va de trois VM éteintes à l'API saine ; Docker inclut le démarrage de son moteur Linux puis des trois conteneurs. Ces durées comprennent l'orchestration et la résolution des contrôles de santé. Le repos est relevé après campagne et redémarrage. (1) Premier palier atteignant 90 % du meilleur débit moyen réussi : repère descriptif, pas un seuil de service acceptable.

Étalonnage

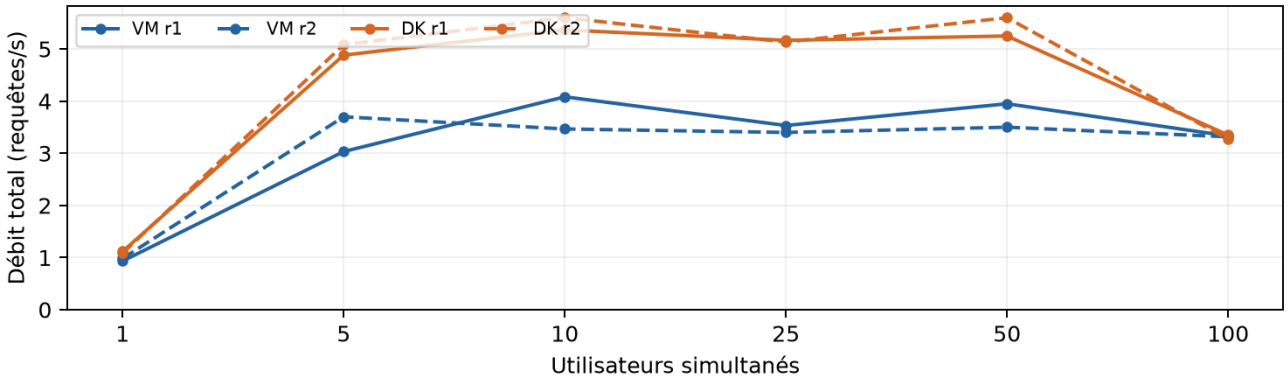
VM : 470 à 736 req/s sur /index.html. DK : 5887 à 6318 req/s sur /index.html. Ce sont des bornes de capacité du couple injecteur/proxy, pas une mesure isolée du plafond de Java. Les métriques de Java et les erreurs sont conservées.

Résultats détaillés

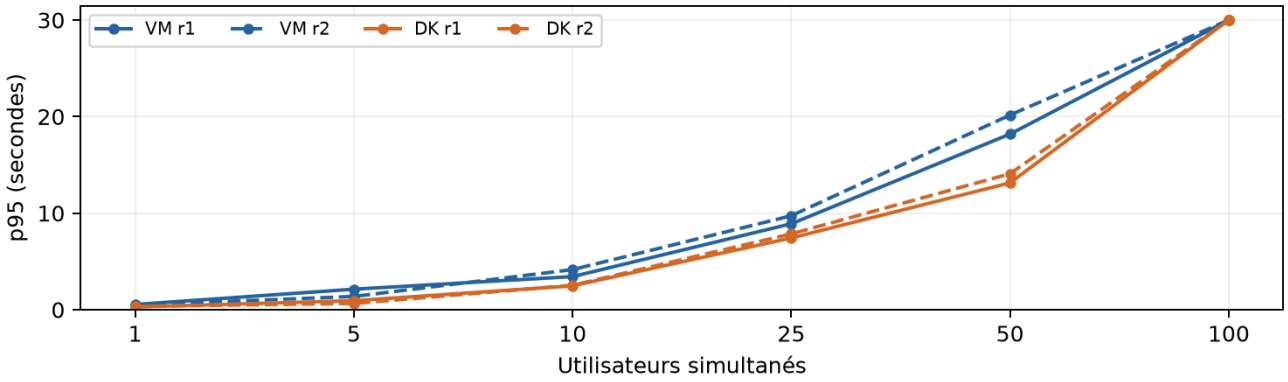
Déploiement	u.	Débit r1 / r2	p95 r1 / r2 (s)	Erreurs r1 / r2 (%)
VM	1	0,93 / 0,98	0,54 / 0,50	0,0 / 0,0
VM	5	3,03 / 3,70	2,13 / 1,37	0,0 / 0,0
VM	10	4,08 / 3,47	3,43 / 4,15	0,0 / 0,0
VM	25	3,53 / 3,40	8,90 / 9,73	0,0 / 0,0
VM	50	3,95 / 3,50	18,19 / 20,16	0,0 / 0,0
VM	100	3,33 / 3,32	30,00 / 30,00	100,0 / 100,0
DK	1	1,12 / 1,08	0,29 / 0,34	0,0 / 0,0
DK	5	4,88 / 5,08	0,93 / 0,65	0,0 / 0,0
DK	10	5,37 / 5,60	2,47 / 2,53	0,0 / 0,0
DK	25	5,17 / 5,13	7,43 / 7,86	0,0 / 0,0
DK	50	5,25 / 5,60	13,14 / 14,08	0,0 / 0,0
DK	100	3,35 / 3,27	30,01 / 30,02	99,0 / 100,0

Écart relatif = |r1 - r2| / moyenne × 100, calculé séparément sur le débit et le p95. Dépassement du seuil de 10 % : VM 5 u., VM 10 u., VM 50 u., DK 1 u., DK 5 u.. Ces paliers restent publiés ; aucun meilleur essai n'est sélectionné pour masquer la variation.

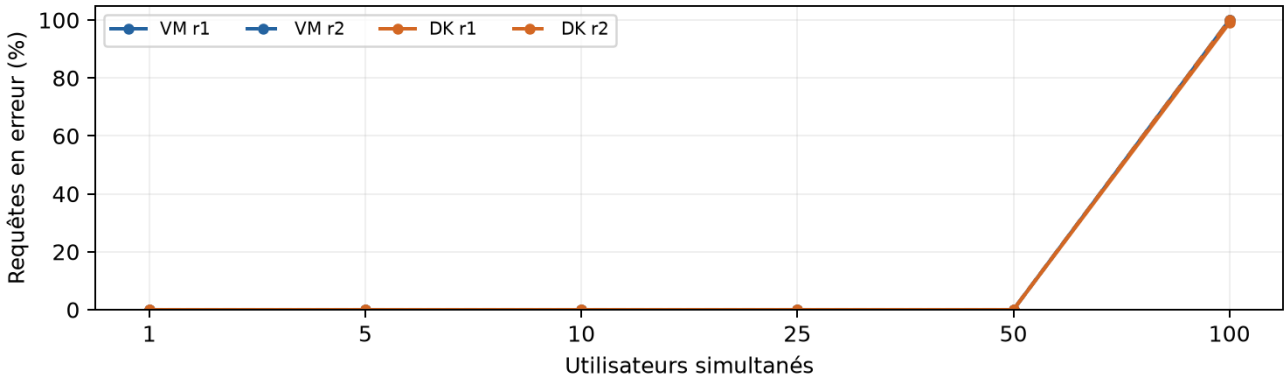
Courbes de la campagne



Débit selon le nombre d'utilisateurs



Latence p95 selon le nombre d'utilisateurs



Taux d'erreurs selon le nombre d'utilisateurs

Écarts chiffrés

Le meilleur débit sans erreur observé vaut 3,77 req/s en VM et 5,48 req/s sous Docker, aux charges indiquées dans la synthèse. À 10 utilisateurs, le p95 moyen est respectivement 3,79 s et 2,50 s. Ces moyennes décrivent deux essais ; ce ne sont pas des garanties de capacité durable.

Explications

Les VM exécutent trois systèmes invités, tandis que les conteneurs partagent le noyau de la VM Linux de Docker Desktop. Le coût de l'ordonnancement et du réseau diffère. Les versions de Python (VM 3.12 / Docker 3.14), MySQL (8.0 / 8.4) et Nginx diffèrent également : les résultats comparent ces installations concrètes, sans isoler l'effet de la virtualisation.

Composant limitant

VM : CPU moyen backend jusqu'à 95,1 % d'un cœur, contre 13,6 % pour MySQL ; maximum 16 connexions, surveillance comprise. Les journaux confirment un pool SQLAlchemy de 5 + 10 connexions épuisé. DK : CPU moyen backend jusqu'à 102,3 % d'un cœur, contre 8,3 % pour MySQL ; maximum 16 connexions, surveillance comprise. Les journaux confirment un pool SQLAlchemy de 5 + 10 connexions épuisé. Le pool est une limite applicative : cela ne prouve pas la saturation du moteur MySQL. Les courbes et les métriques par palier permettent de distinguer le ralentissement préalable de l'échec à forte charge. Après la première surcharge Docker, le backend était unhealthy sans OOM ; un redémarrage hors mesure a précédé la réinitialisation de la série suivante.

Limites

Deux répétitions de 60 s ne couvrent ni l'endurance ni la disponibilité, les pannes ou une distribution multi-hôtes. Le poste héberge aussi l'injecteur ; les tâches de fond, caches et températures ne sont pas parfaitement contrôlés. Les temporisations client plafonnent les latences d'échec autour de 30 s. Les versions et les modalités de quotas/swap diffèrent. Les anciennes mesures sur le PC de Rémi restent historiques et sont exclues de cette comparaison.

Validation et traçabilité

Validation GUI réalisée manuellement par l'utilisateur le 26 octobre 2026. Les captures et relevés montrent HTTP 200 pour la connexion et la consultation nominales, puis HTTP 401 et des assertions échouées avec le mot de passe incorrect. Les preuves sont conservées dans captures/. Traçabilité : configuration.json (empreintes), PROTOCOLE.md, etapes/*-resultat.json,* -serveur.jsonl et *-backend.txt, calibration/, baseline-vm.json et baseline-dk.json. Les CSV bruts vm/dk portant l'identifiant 20261026-152151 sont dans bruts/charge ; l'étalonnage est dans bruts/etalonnage. Les données anciennes restent inchangées.

Autres mesures du rendu

Les mesures de référence de Rémi complètent la partie déploiement : démarrage, mémoire, disque, latence, isolation et persistance. Elles restent distinctes de cette campagne de charge. Voir aussi le rapport historique conservé.

Captures de validation JMeter.

9. Bilan, limites et pistes d’amélioration

Ce que nous avons obtenu

Le projet relie une maquette à une application trois tiers, puis à deux modes de déploiement. Les parcours illustrés montrent la recherche d’un refuge et la persistance d’une publication. Les vérifications réseau montrent les accès permis et les refus attendus. Les volumes Docker conservent la base lors de la recréation des conteneurs.

Sous le protocole de charge commun, Docker atteint un meilleur débit moyen sans erreur que les VM et un p95 inférieur à dix utilisateurs. Les deux installations se dégradent fortement à cent utilisateurs. Cette observation ne permet pas de déclarer Docker universellement plus rapide : les versions et les mécanismes d’exécution diffèrent.

Limites à retenir

Limite	Conséquence pour l’interprétation
Deux répétitions courtes	Pas de conclusion sur l’endurance ou la disponibilité à long terme.
Cinq paliers non répétables à 10 %	Les moyennes doivent être lues avec leurs deux essais, pas comme une capacité garantie.
Périmètres mémoire différents	Pas de gain global de mémoire démontré entre VM et conteneurs.
Versions logicielles différentes	L’effet de la virtualisation n’est pas isolé.
Recette historique distincte des captures récentes	Les deux parcours ne revalident pas toutes les fonctions ni toute la suite automatisée.

Améliorations à tester

La première piste est d’étudier le coût du scénario métier, qui authentifie à chaque itération, et le temps d’occupation des connexions SQLAlchemy. Il faudrait profiler l’API et vérifier les requêtes avant de modifier la taille du pool : augmenter sa limite seule pourrait déplacer le problème vers la base.

Une deuxième campagne devrait fixer les mêmes versions, prévoir davantage de répétitions et séparer l’injecteur du système testé. Des essais longs, des scénarios de lecture et d’écriture distincts et des objectifs de latence explicites permettraient de mieux caractériser un usage réel.

Enfin, un test d’import des OVA sur un poste vierge et une procédure complète de restauration des médias renforceraient la reproductibilité du rendu. Ces points sont des perspectives, pas des validations déjà réalisées.

10. Documents et preuves associés

Le rapport se lit de façon autonome ; les fichiers suivants permettent de retrouver les configurations et les preuves détaillées. Les liens renvoient au dépôt sur la branche remi-dev. Les mesures restent dans leurs formats d'origine.

Élément	Emplacement de référence
Point d'entrée du rendu	docs/RENDU.md
Analyse et règles détaillées	docs/analyse-detaillee.md
Démarche Figma et prompts	docs/demarche-maquette-figma.md
Diagrammes et sources	docs/diagrams/README.md
Exemples de requêtes API	docs/requetes.md
Recette historique	docs/tests.md
Parcours illustrés	docs/captures/parcours-application/README.md
Configuration des VM	deploiement/PROCEDURE.md
Configuration Docker	docker/PROCEDURE.md
Preuves Docker de référence	docs/preuves-docker/2026-10-26/README.md
Comparaison JMeter et bruts	Campagne de 24 essais
Captures JMeter positives et négatives	Validation graphique
Protocole et automatisation	charge/comparaison/README.md
Archives VM téléchargeables	Release GitHub du rendu

Les sujets des quatre parties du projet DEV 1 ont servi de cadre : analyse et développement, machines virtuelles, Docker et JMeter. Les résultats de ce rapport proviennent des documents et mesures du projet ; les valeurs historiques ne sont pas substituées aux nouvelles séries.

Les tableaux de bord HTML des anciennes campagnes JMeter sont conservés dans une sauvegarde locale hors de myhike. Le rendu conserve leurs données brutes utiles et les synthèses Markdown. Le code applicatif, les captures et les configurations restent dans le dépôt.